Московский политехнический университет

Факультет информационных технологий

ОТЧЁТ

по лабораторной работе № 2

«Виртуализация»

по дисциплине «Автоматизация процессов жизненного цикла программных средств»

Выполнили: студенты группы 241-3211

Первухин Р. А., Гурьев С. А., Полончиков С. А.

Проверил: Красников А. С.

Москва

2026

ОГЛАВЛЕНИЕ

1 Цель работы.....3

2 Теоретические сведения.....3

3 Ход выполнения работы.....4

3.1 Установка гипервизора.....4

3.2 Создание виртуальных машин.....5

3.3 Объединение стендов в локальную сеть.....6

3.4 Размещение проекта из удалённого репозитория.....10

3.5 Развёртывание средств разработки и запуск прототипа.....11

4 Результаты проверок.....13

5 Ответы на вопросы для изучения.....14

6 Заключение.....16

Список использованных источников.....17

1 ЦЕЛЬ РАБОТЫ

Цель работы — получить практические навыки создания и настройки виртуальной инфраструктуры для сопровождения программного средства: развернуть три Unix-подобные виртуальные машины, соответствующие стендам TEST, STAGE и PROD, объединить их в единую локальную сеть, разместить на тестовом стенде файлы проекта из удалённого Git-репозитория, установить средства разработки и запустить прототип приложения.

В соответствии с заданием лабораторной работы необходимо:

• установить гипервизор, выбранный студентом;

• создать не менее трёх виртуальных машин с Unix-подобными операционными системами для стендов test, stage и prod соответственно;

• объединить созданные виртуальные машины в единую локальную сеть так, чтобы команда ping работала по принципу «от всех ко всем»;

• разместить на тестовом стенде файлы проекта из удалённого репозитория при помощи утилиты Git;

• развернуть на стендах средства разработки согласно выбранному для проекта стеку; поскольку для проекта выбран скриптовый язык Python, требуется запустить прототип приложения;

• подготовить отчёт с текстовым описанием и скриншотами по перечисленным пунктам.

Объектом автоматизации в рамках дисциплины является информационная система учёта успеваемости студентов StudentLedger, поэтому созданные стенды предназначены для её последующей разработки, проверки и эксплуатации.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Виртуализация — это предоставление набора вычислительных ресурсов или их логического объединения, абстрагированное от аппаратной реализации и обеспечивающее логическую изоляцию вычислительных

процессов, выполняемых на одном физическом ресурсе. Гипервизор предоставляет гостевым системам виртуальное оборудование: процессор, память, диски, сетевые адаптеры. Каждая гостевая операционная система работает со своим ядром и не зависит от соседних гостей.

При разработке программного обеспечения виртуализация даёт следующие преимущества: сокращение временных и трудовых затрат на настройку экземпляров, уменьшение затрат на оборудование, изоляция окружений, возможность быстрого создания тестового окружения с различными операционными системами и быстрое восстановление после сбоев.

Для объединения виртуальных машин в сеть используются разные типы виртуальных сетей. Виртуальная внутренняя сеть (internal network) связывает только гостей и полностью изолирована от хоста и внешней сети. Сеть «только хост» (host-only) дополнительно предоставляет доступ со стороны хост-машины. Сеть типа «мост» (bridge) подключает виртуальный адаптер к физическому адаптеру хоста, и гость становится равноправным узлом локальной сети. Преобразование сетевых адресов (NAT) позволяет гостям выходить во внешнюю сеть через хост, но входящие соединения к ним по умолчанию недоступны.

В данной работе применена комбинация двух типов сетей: NAT используется для доступа гостевых систем во внешнюю сеть (получение пакетов и клонирование репозитория), а общий сегмент «только хост» — для связи стендов между собой по статическим адресам.

3 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Установка гипервизора

В качестве гипервизора выбран Oracle VirtualBox — кроссплатформенное средство виртуализации, распространяемое бесплатно и поддерживающее гостевые системы Ubuntu. Гипервизор установлен на хост-машину под управлением Windows 10 Pro 22H2 (архитектура amd64, 8

логических ядер, 16 ГБ оперативной памяти). Используемая версия — Oracle VirtualBox 7.2.20 r175154.

После установки созданы три виртуальные машины: studentledger-test, studentledger-stage и studentledger-prod. На рисунке 1 приведено окно менеджера VirtualBox с тремя одновременно работающими стендами и параметрами выбранной машины.

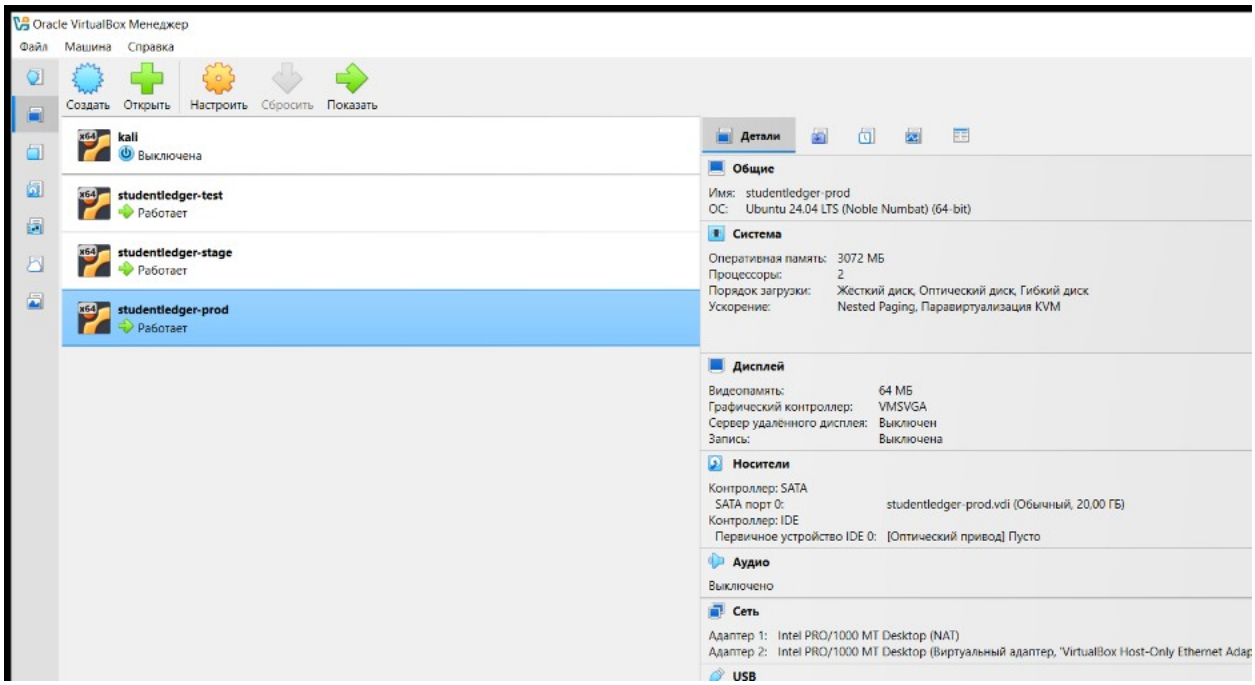


Рисунок 1 — Три работающие виртуальные машины в менеджере Oracle VirtualBox 7.2.20

3.2 Создание виртуальных машин

Для всех стендов использован один и тот же установочный образ Ubuntu Server 24.04.5 LTS (Noble Numbat) для архитектуры amd64. Целостность образа проверена сравнением контрольной суммы SHA-256 с официальным файлом SHA256SUMS проекта Ubuntu; результат сверки сохранён в составе подтверждений.

Каждой машине выделены 2 виртуальных процессора, 3072 МБ оперативной памяти, виртуальный диск VDI объёмом 20 ГБ и 64 МБ видеопамяти. Установка операционной системы выполнена в автоматическом режиме (unattended install на основе cloud-init), после установки имя каждой машины задано командой hostnamectl. Основные параметры стендов приведены в таблице 1.

Таблица 1 — Параметры виртуальных стендов

Стенд	Имя машины	Адрес во внутренней сети	MAC-адрес
TEST	studentledger-test	192.168.56.11/24	08:00:27:51:A0:01
STAGE	studentledger-stage	192.168.56.12/24	08:00:27:51:A0:02
PROD	studentledger-prod	192.168.56.13/24	08:00:27:51:A0:03

На всех трёх стендах установлена Ubuntu 24.04.5 LTS с ядром Linux 6.8.0-146-generic для архитектуры x86-64, а также минимальное графическое окружение Xubuntu (пакет xubuntu-desktop-minimal) и менеджер входа LightDM, что позволило выполнять проверки непосредственно в терминале гостевой системы.

3.3 Объединение стендов в локальную сеть

Каждая виртуальная машина получила два сетевых адаптера. Первый адаптер (Intel PRO/1000 MT Desktop) подключён к сети NAT и обеспечивает выход во внешнюю сеть. Второй адаптер подключён к общему виртуальному сегменту «VirtualBox Host-Only Ethernet Adapter» с подсетью 192.168.56.0/24; адрес хоста в этом сегменте — 192.168.56.1, встроенный DHCP-сервер VirtualBox отключён.

Статические адреса стендов заданы средствами Netplan. В гостевой системе внутренний интерфейс называется slca-internal; его описание сохранено в файле /etc/netplan/60-studentledger.yaml с правами 600. Фрагмент конфигурации для стенда TEST приведён ниже, для STAGE и PROD адрес изменяется на 192.168.56.12 и 192.168.56.13 соответственно.

```
network:
  version: 2
  ethernets:
    slca-internal:
      dhcp4: false
      addresses:
        - 192.168.56.11/24
```

Применение настроек выполнено командой netplan apply, после чего проверены адреса интерфейсов и наличие пакетов утилит git, python3, python3-venv, python3-pip, curl и iputils-ping. На рисунках 2–4 показаны

результаты проверки операционной системы, имени машины, адресов внутреннего сегмента и версий инструментов на каждом стенде.

```
File  Edit  View  Search  Terminal  Help
student@studentledger-test:~$  hostnamectl
 Static hostname: studentledger-test
       Icon name: computer-vm
         Chassis: vm
      Machine ID: fa29038245ae4a2dbbcdadadec5e45cb
         Boot ID: 447ea3f4a4de473c871c0fb25aa702f7
  Virtualization: oracle
Operating System: Ubuntu 24.04.5 LTS
          Kernel: Linux 6.8.0-146-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 10month 2w 2d
student@studentledger-test:~$  ip -br -4 address
lo               UNKNOWN        127.0.0.1/8
enp0s3           UP             10.0.2.15/24 metric 100
slca-internal    UP             192.168.56.11/24
student@studentledger-test:~$  git --version
git version 2.43.0
student@studentledger-test:~$  python3 --version
Python 3.12.3
student@studentledger-test:~$
```

Рисунок 2 — Стенд TEST: операционная система, имя машины, адрес 192.168.56.11, версии Git и Python

```
File  Edit  View  Search  Terminal  Help
student@studentledger-stage:~$       hostnamectl
 Static hostname: studentledger-stage
       Icon name: computer-vm
         Chassis: vm
      Machine ID: b09a1fc44f5445be8395d0308f885a05
         Boot ID: 221f18eac2654bf8bd2cef822280c0d9
  Virtualization: oracle
Operating System: Ubuntu 24.04.5 LTS
          Kernel: Linux 6.8.0-146-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 10month 2w 2d
student@studentledger-stage:~$       ip -br -4 address
lo               UNKNOWN        127.0.0.1/8
enp0s3           UP             10.0.2.15/24 metric 100
slca-internal    UP             192.168.56.12/24
student@studentledger-stage:~$       git --version
git version 2.43.0
student@studentledger-stage:~$       python3 --version
Python 3.12.3
student@studentledger-stage:~$
```

Рисунок 3 — Стенд STAGE: операционная система, имя машины, адрес 192.168.56.12, версии Git и Python

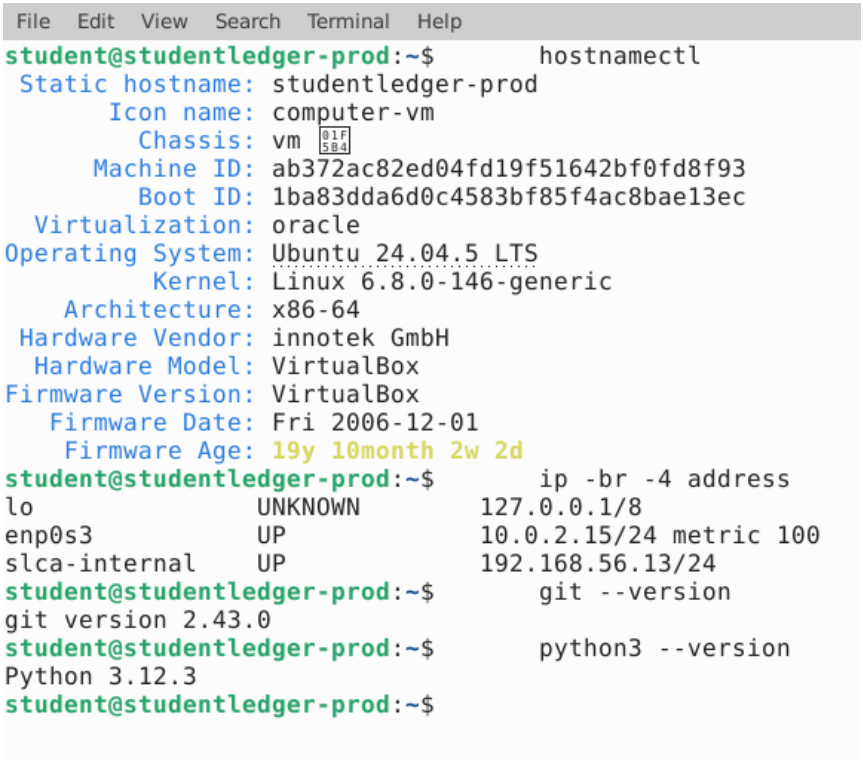


Рисунок 4 — Стенд PROD: операционная система, имя машины, адрес 192.168.56.13, версии Git и Python

Связность стендов проверена командой ping в шести направлениях: с каждого стенда на два других. В каждом сеансе передавалось по четыре пакета протокола ICMP. Во всех направлениях получены ответы от всех четырёх пакетов, потери составили 0 %. Результаты проверок приведены на рисунках 5–7.

```
File  Edit  View  Search  Terminal  Help
student@studentledger-test:~$  ping -c 4 -W 2 192.168.56.12
PING 192.168.56.12 (192.168.56.12) 56(84) bytes of data.
64 bytes from 192.168.56.12: icmp_seq=1 ttl=64 time=0.383 ms
64 bytes from 192.168.56.12: icmp_seq=2 ttl=64 time=0.634 ms
64 bytes from 192.168.56.12: icmp_seq=3 ttl=64 time=0.763 ms
64 bytes from 192.168.56.12: icmp_seq=4 ttl=64 time=0.342 ms

--- 192.168.56.12 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3103ms
rtt min/avg/max/mdev = 0.342/0.530/0.763/0.174 ms
student@studentledger-test:~$  ping -c 4 -W 2 192.168.56.13
PING 192.168.56.13 (192.168.56.13) 56(84) bytes of data.
64 bytes from 192.168.56.13: icmp_seq=1 ttl=64 time=0.779 ms
64 bytes from 192.168.56.13: icmp_seq=2 ttl=64 time=0.500 ms
64 bytes from 192.168.56.13: icmp_seq=3 ttl=64 time=0.516 ms
64 bytes from 192.168.56.13: icmp_seq=4 ttl=64 time=0.459 ms

--- 192.168.56.13 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3183ms
rtt min/avg/max/mdev = 0.459/0.563/0.779/0.126 ms
student@studentledger-test:~$
```

Рисунок 5 — Проверка связи со стенда TEST: ping до STAGE (192.168.56.12) и PROD (192.168.56.13)

```
File  Edit  View  Search  Terminal  Help
student@studentledger-stage:~$       ping -c 4 -W 2 192.168.56.11
PING 192.168.56.11 (192.168.56.11) 56(84) bytes of data.
64 bytes from 192.168.56.11: icmp_seq=1 ttl=64 time=0.253 ms
64 bytes from 192.168.56.11: icmp_seq=2 ttl=64 time=0.278 ms
64 bytes from 192.168.56.11: icmp_seq=3 ttl=64 time=0.219 ms
64 bytes from 192.168.56.11: icmp_seq=4 ttl=64 time=0.788 ms

--- 192.168.56.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3156ms
rtt min/avg/max/mdev = 0.219/0.384/0.788/0.233 ms
student@studentledger-stage:~$       ping -c 4 -W 2 192.168.56.13
PING 192.168.56.13 (192.168.56.13) 56(84) bytes of data.
64 bytes from 192.168.56.13: icmp_seq=1 ttl=64 time=0.549 ms
64 bytes from 192.168.56.13: icmp_seq=2 ttl=64 time=0.519 ms
64 bytes from 192.168.56.13: icmp_seq=3 ttl=64 time=0.255 ms
64 bytes from 192.168.56.13: icmp_seq=4 ttl=64 time=0.447 ms

--- 192.168.56.13 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3108ms
rtt min/avg/max/mdev = 0.255/0.442/0.549/0.114 ms
student@studentledger-stage:~$
```

Рисунок 6 — Проверка связи со стенда STAGE: ping до TEST (192.168.56.11) и PROD (192.168.56.13)

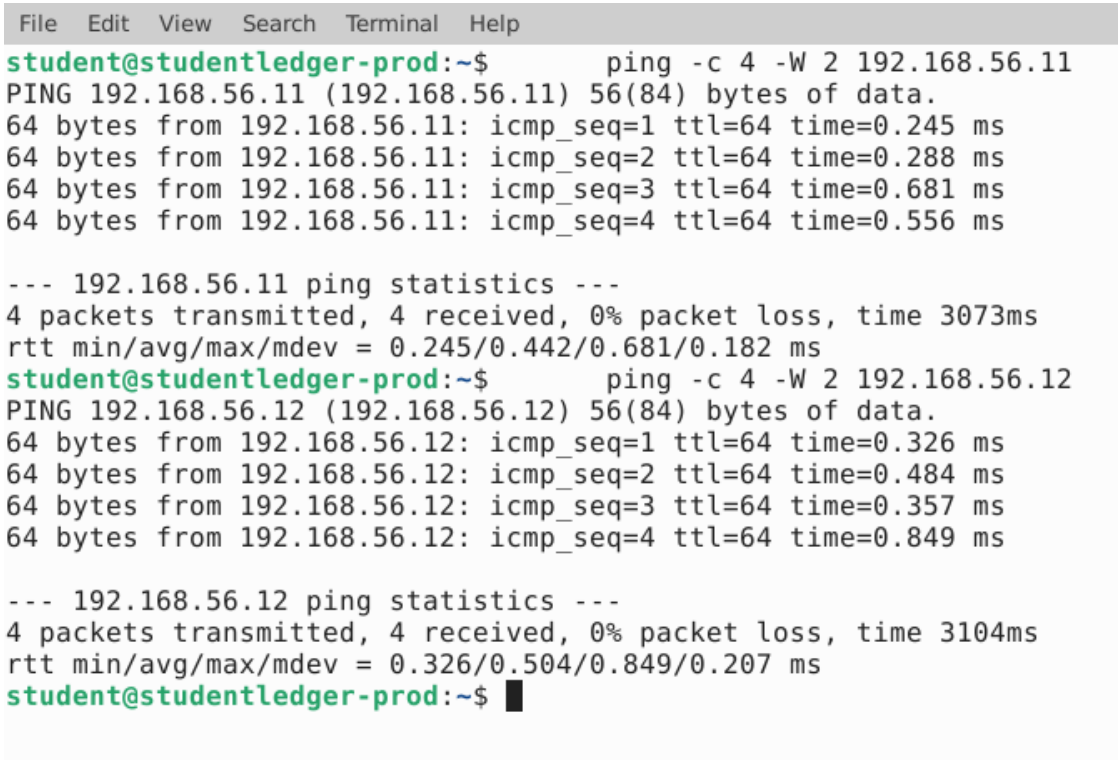


Рисунок 7 — Проверка связи со стенда PROD: ping до TEST (192.168.56.11) и STAGE (192.168.56.12)

3.4 Размещение проекта из удалённого репозитория

Исходный код проекта размещён в удалённом репозитории https://github.com/PervuhinRoman/SLCA.git. На тестовом стенде выполнено клонирование основной ветки репозитория, после чего проверены настроенный удалённый репозиторий, текущая ветка и хеш коммита.

```
git clone --branch master
https://github.com/PervuhinRoman/SLCA.git
cd StudentLedger
git remote -v
git branch --show-current
git rev-parse HEAD
```

Клонирование выполнено успешно: настроен удалённый репозиторий origin, рабочая ветка — master, зафиксированный хеш коммита — 816ac3eca4fcf73d2de12a65b907916a97fcaeee. Результат показан на рисунке 8. Тот же порядок действий применён и на стендах STAGE и PROD, что обеспечивает одинаковый набор файлов проекта на всех стендах.

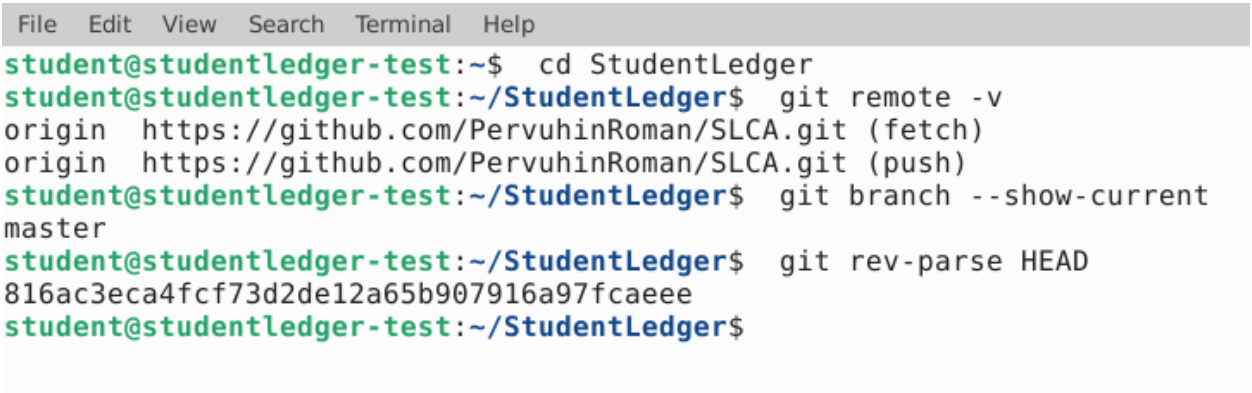


Рисунок 8 — Клонированный репозиторий проекта на стенде TEST: адрес удалённого репозитория, ветка master и хеш коммита

3.5 Развёртывание средств разработки и запуск прототипа

Согласно техническому заданию проект разрабатывается на языке Python с использованием веб-фреймворка FastAPI. На стендах установлены Python 3.12.3, менеджер пакетов pip и модуль venv, после чего в каталоге проекта создано изолированное виртуальное окружение и установлены закреплённые зависимости проекта из файла requirements.txt. Проверка согласованности зависимостей командой pip check не выявила нарушенных требований.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m pip check
.venv/bin/python -m uvicorn app.main:app --host
192.168.56.11 --port 8000
```

Прототип приложения реализует единственную точку проверки работоспособности GET /health, возвращающую JSON-документ {"status": "ok"}. Запуск приложения на тестовом стенде автоматизирован службой systemd studentledger.service: она включена в автозапуск (enabled) и находится в активном состоянии (active), а приложение прослушивает внутренний адрес стенда 192.168.56.11:8000, поэтому доступно с других стендов внутренней сети, но не из внешней сети.

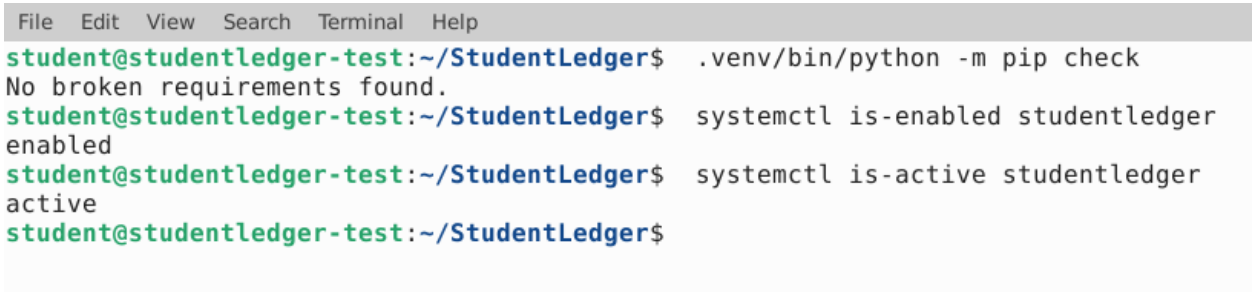


Рисунок 9 — Стенд TEST: проверка зависимостей и состояние службы studentledger

Работоспособность приложения проверена запросом к точке /health с заголовками ответа. Сервер вернул код HTTP 200 (OK), тип содержимого application/json и тело {"status": "ok"}, что подтверждает успешный запуск прототипа на тестовом стенде (рисунок 10).

```
curl -i --max-time 5 http://192.168.56.11:8000/health
```

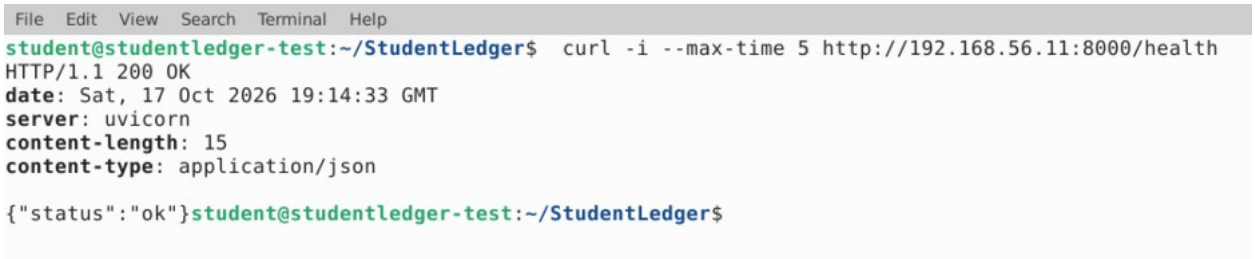


Рисунок 10 — Ответ прототипа на тестовом стенде: HTTP/1.1 200 OK и тело {"status": "ok"}

4 РЕЗУЛЬТАТЫ ПРОВЕРОК

Сводные результаты выполнения пунктов задания приведены в таблице 2. Все проверки выполнены на реально работающих виртуальных машинах, скриншоты сняты в гостевых системах 17.10.2026.

Таблица 2 — Сводные результаты проверок

Проверка	Команда или средство	Результат	Рисунок
Гипервизор установлен, три ВМ работают	Oracle VirtualBox 7.2.20	Три машины в состоянии «Работает»	1
Единая операционная система стендов	hostnamectl, /etc/os-release	Ubuntu 24.04.5 LTS, ядро 6.8.0-146, x86-64	2–4
Статические адреса внутреннего сегмента	ip -br -4 address	192.168.56.11, .12, .13 (маска /24)	2–4
Средства разработки на всех стендах	git --version, python3 --version	Git 2.43.0, Python 3.12.3	2–4
Связность «от всех ко всем»	ping -c 4 -W 2	6 из 6 направлений, потери 0 %	5–7
Проект из удалённого репозитория на TEST	git clone, git rev-parse	origin SLCA, ветка master, коммит 816ac3e	8
Зависимости проекта	pip install -r requirements.txt, pip check	No broken requirements found	9
Автозапуск прототипа	systemctl is-enabled / is-active	enabled, active	9
Точка проверки работоспособности	curl -i /health	HTTP 200, {"status": "ok"}	10

Таким образом, все пункты задания лабораторной работы выполнены: созданы три виртуальные машины с Unix-подобной операционной системой, объединённые в общую локальную сеть, на тестовом стенде размещён проект из удалённого репозитория, установлены средства разработки и запущен прототип приложения, отвечающий на запросы кодом 200.

5 ОТВЕТЫ НА ВОПРОСЫ ДЛЯ ИЗУЧЕНИЯ

Вопрос 1. Что общего и чем отличаются различные типы сетей для виртуальных машин (virtual network, bridge, nat)?

Общее: во всех случаях гипервизор создаёт программный сетевой адаптер и виртуальный коммутатор, поэтому гостевые системы обмениваются кадрами так же, как физические компьютеры, и не требуют отдельных драйверов оборудования. Отличия состоят в том, с чем соединён этот коммутатор и кто выполняет преобразование адресов. Внутренняя виртуальная сеть связывает только гостей одного гипервизора и изолирована от хоста и внешней сети — подходит для закрытых испытаний. Сеть «только хост» добавляет к этому интерфейс на хост-машине, поэтому гость доступен с хоста, но не из локальной сети. Сеть типа «мост» подключает виртуальный адаптер к физическому адаптеру хоста: гость получает адрес в физической сети и становится её равноправным участником, доступным извне. В режиме NAT гипервизор подменяет адреса гостей адресом хоста, поэтому гости выходят во внешнюю сеть, но входящие соединения к ним требуют отдельной настройки проброса портов, а адреса гостей не видны снаружи.

Вопрос 2. Каким образом осуществляется обработка системных вызовов Unix в виртуальных машинах при их развёртывании в ОС Windows?

При полной виртуализации в гостевой системе работает собственное ядро Unix-подобной операционной системы, поэтому системный вызов, например чтение файла или создание процесса, обрабатывается этим ядром, а не ядром Windows. Гипервизор предоставляет гостю виртуальный процессор с аппаратной поддержкой виртуализации (Intel VT-x или AMD-V) и перехватывает только привилегированные операции и часть прерываний, которые ядро гостя не имеет права выполнять напрямую. Таким образом, каждый системный вызов обслуживается внутри гостя, хост-система не транслирует его в интерфейс Win32 API и не участвует в его обработке; Windows лишь предоставляет виртуальное оборудование и планирует

выполнение гостевого кода. Драйверы гипервизора и гостевые дополнения обеспечивают обмен с хостом (например, общий буфер обмена и общие папки), но не подменяют системные вызовы гостя.

Вопрос 3. Как размещаются на жёстком диске хост-машины виртуальные жёсткие диски виртуальной машины, если их файловые системы различны?

Виртуальный диск представляется гостю как обычное блочное устройство, а на диске хоста он хранится в виде одного файла (в VirtualBox — файла формата VDI) либо набора файлов снапшотов. Внутри этого файла размещается образ блочного устройства: таблица разделов и файловая система гостя, например ext4. Хост-операционная система работает с файлом виртуального диска средствами своей собственной файловой системы (в рассматриваемой конфигурации — NTFS) и не разбирает структуру ext4: она лишь выделяет файлу место и выполняет операции чтения и записи блоков. Поэтому различие файловых систем гостя и хоста не создаёт проблемы — файловая система гостя интерпретируется только гостевым ядром, а для хоста виртуальный диск остаётся обычным файлом, размер которого может увеличиваться по мере заполнения (динамический диск) или быть задан заранее (фиксированный диск).

6 ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы освоены практические приёмы создания и настройки виртуальной инфраструктуры. Установлен гипервизор Oracle VirtualBox 7.2.20, созданы три виртуальные машины с Ubuntu 24.04.5 LTS, соответствующие стендам TEST, STAGE и PROD, и заданы их ресурсы.

Все стенды объединены в общий сегмент 192.168.56.0/24 со статическими адресами; проверка связи в шести направлениях показала отсутствие потерь пакетов. На тестовом стенде размещён проект из удалённого репозитория SLCA, установлены средства разработки (Git, Python 3.12, модуль venv, зависимости проекта), запущен прототип приложения FastAPI, который отвечает на запрос GET /health кодом HTTP 200 и документом {"status": "ok"}.

Полученные стенды образуют основу для выполнения последующих лабораторных работ: контейнеризации приложения, настройки непрерывной интеграции и доставки, статического анализа, автоматизированного тестирования и миграций базы данных. Все результаты подтверждены скриншотами гостевых систем и сохранены в составе подтверждений к отчёту.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторные работы по дисциплине «Автоматизация процессов жизненного цикла программных средств»: лабораторная работа № 2 «Виртуализация». — Москва: Московский политех, 2024. — С. 5–7.

2. Техническое задание на создание автоматизированной системы «Информационная система учёта успеваемости студентов» (лабораторная работа № 1). — Москва: Московский политех, 2026.

3. Oracle VM VirtualBox: User Manual [Электронный ресурс]. — URL: https://www.virtualbox.org/manual/ (дата обращения: 17.10.2026).

4. Ubuntu Server documentation: Configuring networks [Электронный ресурс]. — URL: https://documentation.ubuntu.com/server/explanation/networking/configuring-networks/ (дата обращения: 17.10.2026).

5. FastAPI documentation: First steps [Электронный ресурс]. — URL: https://fastapi.tiangolo.com/tutorial/first-steps/ (дата обращения: 17.10.2026).